

Universidade Estadual de Maringá - Departamento de Informática
Linguagens de Programação
Prof. Marcelo Malheiros

ESPECIFICAÇÃO DO SEGUNDO TRABALHO

Os objetivos são dois:

1. Adaptar o **analisador** implementado anteriormente para gerar um *bytecode* simples que permita a execução posterior de um **programa válido**. Esse *bytecode* pode ser gerado para a saída padrão ou gravado em um arquivo.
2. Implementar um interpretador para executar o *bytecode* gerado, de forma a produzir a saída esperada do programa original (esta saída deve ser idêntica à execução pelo compilador oficial da linguagem escolhida).

Instruções mínimas do *bytecode* (outras podem ser adicionadas ao seu critério):

1. Delimitação da implementação de função → `fun f()` e `ret`
2. Atribuição de constante inteira a variável (com escopo explícito) → `set f():b 5`
3. Comando de saída simples (a definir)
4. Chamada de funções → `sub g()`

Funcionalidades obrigatórias do analisador:

- O interpretador **deve** detectar eventuais erros em tempo de execução.
 - Com uma linguagem simples e um analisador correto eventualmente o *bytecode* gerado sempre estará certo, mas por segurança o interpretador deve indicar funções e variáveis não encontradas em *runtime*, para auxiliar no desenvolvimento.
- As instruções de *bytecode* de saída **devem** gerar resultados na saída padrão.

Roteiro sugerido para a implementação:

1. Ajustar o analisador conforme a implementação de referência no AVA (V5), que emite *bytecode* e os agrupa agrupa por corpo de função.
2. Usar o *explainer* disponível em <https://mgmalheiros.github.io/explainer/stack-frame/> para compreender o processo de interpretação (*call stack* e estrutura de cada *activation frame*).
3. Implementar o interpretador:
 - a. ler o *bytecode* para uma estrutura de dados auxiliar, indexando cada linha;
 - b. mapear os pontos de entrada de todas as funções;
 - c. iniciar a execução a partir da “função” `global`.
4. Conferir para cada programa válido (e eventualmente outros) se a saída produzida coincide com o compilador de referência.
5. Documentar em arquivo **.txt** ou **.md** detalhes importantes sobre a implementação.
6. Se desejado, é possível adicionar novas funcionalidades no analisador (adicionando novos programas de teste).

Funcionalidades **opcionais** do analisador (as mesmas do primeiro trabalho):

- Gerar advertências da linguagem, como variáveis não utilizadas ou sombreamento (*shadowing*) de variáveis de mesmo nome e definidas em um escopo mais externo.
- Detectar erros associados a funções, usando também a tabela de símbolos, para identificar funções duplamente definidas ou desconhecidas.
- Reconhecer e declarar como variáveis locais os parâmetros de uma função.
- Identificar funções declaradas como retornando valores e verificar se as mesmas contém pelo menos um comando de retorno (*return*).
- Verificar se funções com retorno são chamadas como parte do lado direito de uma atribuição. E da mesma forma, verificar se funções sem retorno são chamadas como comandos (*statements*).

Avaliação:

- O trabalho pode ser feito individualmente ou em dupla (par) de duas (2) pessoas.
- A validação do projeto será feita manualmente pelo professor, testando cada exemplo fornecido contra o analisador submetido e então sendo executado o interpretador.
- Na quarta avaliação da disciplina (prova escrita) haverá questões conceituais sobre a implementação do analisador, a geração de código e o processo de interpretação.